CNN

CNN is a deep learning model which was initially designed to work with 1D, 2D and 3D data

CNNs are particularly well-suited for data that has a **spatial (temporal) locality relationship**.
→ This characteristics is most evident in tasks involving images, videos and even audio or time series data, where the relationship between nearby elements is crucial for understanding the overall structure and content of the data

CNNs are used mainly for:

- Image processing
- Classifications
- Segmentation
- Object Detections
- Auto correlated data

Architecture: Each input image will pass through a series of Convolution layers

- Filters (Kernels)
- Pooling
- Fully connected layers (FC)
- Apply Softmax function to classify an object (0 and 1)

CNN Layers

To compute the pixel value i of a r-th feature map for layer l, we use:

$$x_i^{(l)} = g\left(\sum_{j=1}^{k} x_j^{(l-1)} * w_j^{(l-1)} + b_j^{(l-1)}\right)$$

- $g(\,)$ is an activation function
- $x^{(l-1)}$ is 1 of the input regions
- $w^{(l-1)}$ is the filter with size of $N \times N$
- b_i is the bias
- k is the total pixel of filter
- * is the elements-wise multiplication

Consider a 2D input feature map, $x^{(l-1)}$ with $size = (d, d) = (5, 5)$
Parameters:

- $w^{(l-1)}$ filter's size = $(N, N) = (3, 3)$
- Stride $s = 1$ (number of pixels shifts over the input matrix
- Padding = no

$x^{(l)}$ feature map's output size = $\frac{d-N}{s} + 1 = 3$

N number of filters produces N number of feature maps

Weights Sharing

This happens across the **receptive field** of the neurons (filters) in a particular layer. Weights are the numbers within each filter

So essentially we are trying to **learn a filter**. These filters act on a certain receptive field (small section) of the image

When the filter moves through the image, the filter does not change
→ If an edge is important to learn in a particular part of an image, it is important in other parts of the image too

Weight sharing is to provide **translation invariance**: Allows the network to recognize the same object in an image no matter where it appears

Sparsity of Connection

This means that each element of the output depends only on the small section of the input

For example, an element in the output of convolutional layer with 3×3 *filter will depend only on 9 numbers from the input*

As we get deeper into the network, the outputs will depend on less and less numbers

This allows us to **train with less samples** and **prevent overfitting**

Activation Function

Neural networks use **non-linear activation functions**, which can help the network learn complex data, calculate and learn almost any function representation a question and provide accurate predictions

Sigmoid and Tanh activation functions

Sigmoid: $g(z) = \frac{1}{1+e^{-z}}$

Tanh: $g(z) = tanh(z)$

Limitations:

- **Limited sensitivity and saturation of the function**
  - Large values snap to 1 and small values snap to -1 or 0
  - The functions are only really sensitive to changes around their mid-point of their input, such as 0.5 for sigmoid and 0.0 for tanh
- **Vanishing gradient**
  In a feedforward network, the back propagated error typically decreases exponentially as a function of the distance from the final layer, resulting in the model impossible learn
- **Computationally expensive**

ReLU

Rectified Linear Unit

$f(x) = max(0, x)$

ReLUs are nearly linear

- They preserve many of the properties that make linear models easy to optimize with gradient-based methods
- They also preserve many of the properties that make linear models generalize well

Layers

Pooling Layer

Pooling is a technique used to **reduce the spatial size (width and height) of a feature map** while keeping the most important information.

Pooling mainly helps to:

- **Reduce computation** by making feature maps smaller.
- **Reduce overfitting** by removing some unnecessary details.
- **Keep important features** while making the network less sensitive to their exact position.

Types of Pooling: Max, Average, Min, …

Fully Connected Layer

In a FC layer, each neuron is connected to **every** neuron in the previous layer, and **each connection has its own weight**, as seen in regular ANN

In contrast, in a Convolutional layer, each neuron is only connected to **a few** nearby (local) neurons in the previous layer, and **the same set of weights** (and local connection layout) is used for every neuron

Benefit of replacing FC with Convolutional layer: The number of parameters to adjust are reduced due to the fact that the weights are shared in a Convolutional layer
→ Faster and more robust learning

Softmax Layer

Softmax layer is typically the final output layer in a neural network that performs multi-class classification (like object recognition)

The Softmax function takes a vector $z = [z_1, z_2, \dots, z_k]$ of k output classes and maps them to a probability distribution, with each value in the **range** $(0, 1)$ and **all the values summing to 1**

$$h_\theta^{(c)}(x) = softmax(z_c) = \frac{e^{zc}}{\sum_{i=1}^{k} e^{zi}} \quad \text{where} \quad 1 \le i \le k$$

Optimization Algorithms

w is often chosen to be updated based on mini-batch GD

Because the loss function of Neural Networks are complex and non-convex, we may use:

- **SGD with Momentum**: This helps accelerate SGD in the relevant direction and dampens oscillations by adding a fraction of the update vector of the past step to the current update vector
    - In short: Uses accumulated past gradients to guide direction → Faster convergence
    - **Easy analogy:** A ball rolling downhill gains momentum.
- **Nesterov Accelerated Gradient (NAG):** Similar to momentum, but the gradient is calculated at the position ahead in the direction of the momentum. This anticipatory update prevents overshooting and improves convergence
    - In short: Computes gradient at a look-ahead position → Reduces overshooting
    - **Easy analogy:**
      Momentum says: "I'm going this way!"
      NAG says: "I'm going this way, but let me check what's ahead first."

- **Adagrad:** This adapts the learning rate to the parameters, performing smaller updates for parameters associated with frequently occurring features, and larger updates for parameters associated with infrequent features. It is particularly useful for sparse data
    - In short: Adapts learning rate using gradient history: Large past updates → Smaller future steps (already updated enough, so slow down that weight)
    - **Easy analogy:** 1.0 → 0.5 → 0.2 → 0.05 → 0.001 → ...
    - **Problem:** Adagrad remembers all past gradients. Therefore, its learning rate can keep decreasing. Eventually, learning can become extremely slow.
- **RMSprop:**
    - Instead of remembering all past gradients, it focuses more on recent gradients.
    - This adjusts the learning rate by dividing by an exponentially decaying average of squared gradients.
    - This allows it to adapt the learning rate for each of the parameters
- **Adam (Adaptive Moment Estimation):**
    - Combines elements of momentum and RMSprop, maintaining a moving average of both the gradients and their squared values to adjust the learning rate for each parameter.
    - Adam is widely used due to its high performance in a variety of problems
- **Adedelta:**
    - Instead of accumulating all historical gradients, Adadelta uses a moving average, so **recent gradients matter more**
    - An extension of Adagrad that seeks to reduce its aggressive, monotonically decreasing learning rate by calculating a window of accumulated past gradients

CNN Architectures

LeNet-5
AlexNet
VGG
GoogleNet
ResNet:

- Solve Vanish Gradient: Neglecting earlier layers
- Solve Curse of Dimensionality: Causes degradation problem. Shallower networks sometimes learn better than their deeper counterparts

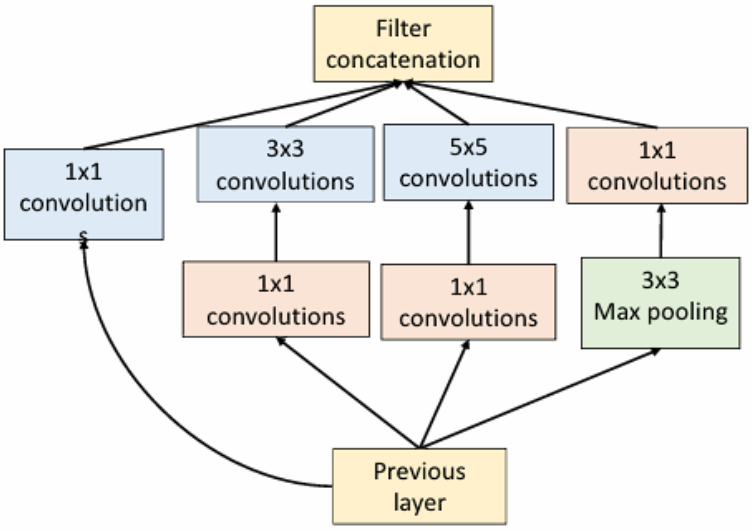
Dropout

Individual nodes are either dropped out of the net with a $1 - p$ probability or retained with a p probability

The inbound and outbound edges to a dropped-out node are also removed

Inception Layer

This is a CNN module combining parallel convolutions of various filter sizes and pooling, concatenated to capture features at multiple scales



Intuition: Allow the internal layers to pick and choose which filter size will be relevant to learn the required information

Skip connections

ResNet uses Skip connections (residual connections):

- This allows gradients to flow through a network directly, bypassing multiple layers without attenuation during backpropagation
- These connections add the output from an earlier layer to a later layer, effectively creating shorter paths for the gradient signal and thus mitigating the vanishing gradient

Batch Normalisation

BN is adopted right after each convolution and before activation

BN is used to encounter the internal covariance shift caused by the changes in the distribution of the layer's input

Problems

1. Overfitting

Definition:

- A model that can fit well to the training data but not generalized to new and unseen testing data.
- Low bias, high variance

Normal causes:

- Model is too complex
- Model was trained with a small amount of training

Solution:

- Gather more training data
- Dataset augmentation (Translation, flip, mirror, noise, colour augmentation, …)
- Reduce complexity (Increase number of neurons, layers, weights parameters)
- Regularization (L1 / L2, dropout, early stoping)

2. Underfitting

Definition:

- A model that performs poorly on both the training and testing data
- High bias, low variance

Normal causes:

- Model is too simple

Solution:

- Increase complexity (Add new layers, increase number of neurons)
- Increase training time
- Reduce dropout

3. Gradient Exploding

Definition:

- This is a problem in training DNNs where gradients can grow exponentially during backpropagation, causing very large weight updates and leading to numerical instability
- This results in model training failure, with weights becoming NaN or diverging to infinity

Solution:

- Gradient Clipping (forcing the gradient values to a specific range)
- Smaller learning rate
- Weight regularization
- Data normalization
- Fewer layers

4. Gradient Vanishing

Definition:

- Occurs when gradients become increasingly small during backpropagation, causing weights in earlier layers to barely update

Solution:

- **Use ReLU** (If use Sigmoid, it squishes a large input space into a small input space between 0 and 1. Therefore, a large change in the input of the sigmoid function will cause a small change in the output → The derivative becomes small)
- **Use residual based network** (It provides residual connections straight to earlier layers
- **Batch normalization** (It forces the activations of a layer to follow a single distribution, independent of the changes in the parameters of upstream layers
- **Combine careful initialization of weights**